



VERDAD, BELLEZA, PROSPERIDAD

FI FACULTAD DE
INGENIERIA
TAMPICO

Coordinación de Carrera
EXAMEN DEPARTAMENTAL
R-OP-33-06-02

Materia: **Fundamentos de Programación**

Periodo Escolar: **2026-1**

Nombre del Alumno: Jesus Perter Carzaco

Grado y Grupo: 3 - N

Calif.: _____

Problema: Sistema de Calificaciones Matricial

Se requiere construir un programa de captura de calificaciones de N número de alumnos, de 2 asignaturas. El programa debe solicitar el número de alumnos, debe usar una matriz para guardar el nombre del alumno y la calificación de las dos asignaturas.

Para garantizar la integridad de los datos, el alumno debe programar la siguiente estructura modular:

- Una función que reciba una cadena de texto y verifique que la entrada **no esté vacía** (retornando un booleano).
- Una función que reciba una cadena de texto y valide que contenga exclusivamente un número entero de **1 o 2 cifras** (retornando un booleano), uds deben indicar cual será si la calificación es 1 al 10 o del 1 al 100.
- La captura de cada celda de la matriz debe repetirse indefinidamente si está vacío el dato, o la calificación no es entera positiva y está en el rango definido por el programador.
- Una función independiente que reciba la matriz ya validada y calcule el **promedio general** de todo el grupo, retornando dicho valor.
- Una función que reciba la fila del alumno ya validada y calcule el **promedio del alumno**.
- Deberá después de capturar mostrar todos los nombres de alumnos alineados con las calificaciones de las dos asignaturas y el promedio.
- Desplegará al final el promedio de cada asignatura.

Diseño del Algoritmo 5 pts.

Implementación en Java 5 pts.



24 de abril de 2024
Versión N° 4

algoritmo

Algoritmo ejercicio Final

Definir N, i como entero
Definir nombre como Cadena
Definir calif1, calif2 como Real
Definir Valido como Logico

Repetir

Escribir "Ingreso el numero de alumnos: "

Leer N

Si $N \leq 0$ Entonces

Escribir "Error: el numero de alumnos debe ser mayor a 0"

FinSi

Hasta que $N > 0$

Dimencionar memoria $[N]$

Dimencionar nota1 $[N]$

Dimencionar nota2 $[N]$

Dimencionar Promedio Alumnos $[N]$

Para $i \leftarrow$ hasta N con Paso 1 hacer

Escribir "---Alumno: ", i , "----"

Repetir

Escribir "Nombre: "

Leer Nombre

Valido $\leftarrow$ (Longitud(nombre) > 0)

Si no valido. Entonces

Escribir "Error: el nombre no puede estar vacio"

FinSi

Hasta que valido

nombre $[i] \leftarrow$ calif1

Repetir

Escribir "Calificación 1 (1-100)"

Java

```
Leer Celis 1
Valido <- (celis 1 >= 1 Y celis 1 <= 100)
Si No Valido Entonces
  Escribir "Error, debe ser entre 1 y 100"
FinSi
Hasta que Valido
  nota 1[i] <- celis 1
  ↓
  leer nota 2
  Promedio alumno[i] <- (nota1[i] + nota2[i]) / 2.0
Fin Para

Escribir "Resultados:"
Escribir "Alumno", "nota1", "nota2", "Promedio"

Para i <- 1 hasta N con paso 1 hacer
  Escribir nombre[i], "nota1[i]", "nota2[i]", "Promedio[i]"
Fin Para

Escribir "Promedio General del grupo:", PromedioGeneral(PromedioAlumno, N)
Escribir "Promedio Asignatura 1:", PromedioAsignatura(nota1, N)
Escribir "Promedio Asignatura 2:", PromedioAsignatura(nota2, N)
Fin Algoritmo

Funcion prom <- PromedioGeneral(Promedio, N)
Definir suma, prom como reales
Definir i como entero
suma <- 0
Para i <- 1 hasta N con paso 1 hacer
  suma <- suma + Promedio[i]
Fin Para
prom <- suma / N
Fin Funcion
```

Java

Java

```
import java.io.*;
public class EjercicioFinal {
    public class static void main (String [] args) throws IOException {
        BufferedReader reader = new BufferedReader (new InputStreamReader (System.in));
        int N;
        String nombre;
        double calif1, calif2;
        do {
            System.out.println ("Ingrese el numero de alumnos");
            try {
                N = Integer.parseInt (reader.readLine());
            } catch (NumberFormatException e) {
                N = 0;
            }
        } while (N <= 0);
        String [] nombres = new String [N];
        double [] nota1 = new double [N];
        double [] nota2 = new double [N];
        double [] promedioAlumnos [N];
        for (int i = 0; i < N; i++) {
            System.out.println ("\n --- Alumno " + (i + 1) + " ---");
            do {
                System.out.println ("Nombre: ");
                nombre = reader.readLine();
                valido = (nombre.length() > 0);
            } while (!valido);
            System.out.println ("Error: El nombre no puede estar vacio");
            nombres [i] = nombre;
            do {
                System.out.println ("Calificación 1 (1-100): ");
            } while (true);
            calif1 = Double.parseDouble (reader.readLine());
            valido = (calif1 >= 1 && calif1 <= 100);
            if (!valido) {
                System.out.println ("Error: Calificación no válida");
            }
        }
    }
}
```

}


```
valido = false;  
}
```

```
if (!valido) {
```

```
    cout << "Error: debe ser un numero entre 1 y 100";
```

```
    while (!valido);
```

```
    nota2[i] = Calif2;
```

```
    promedio Alumno[i] = (nota1[i] + nota2[i]) / 2.0;
```

```
    cout << "\n -----";
```

```
    cout << " Resultados ";
```

```
    cout << "\n -----";
```

```
    cout << "Alumno /+ Nota 1 /+ Nota 2 /+ Promedio";
```

```
    cout << "\n -----";
```

```
    for (int i=0; i<N; i++) {
```

```
        cout << nombres[i] << "\t" << nota1[i] << "\t" << nota2[i] << "\t" << prom[i];
```

```
    }
```

```
    cout << "Promedio General del grupo: ", promedioGeneral(promedioAlumnos, N);
```

```
    cout << "Promedio de asignatura 1: ", promedioAsignatura(nota1, N);
```

```
    cout << "Promedio de asignatura 2: ", promedioAsignatura(nota2, N);
```

```
    public static double promedioGeneral(double[] promedio, int N) {
```

```
        double suma = 0;
```

```
        for (int i=0; i<N; i++) {
```

```
            suma += promedio[i];
```

```
        }
```

```
        return suma / N;
```

```
    }
```

```
    public static double promedioAsignatura(double[] notas, int N) {
```

```
        double suma = 0;
```

```
        for (int i=0; i<N; i++) {
```

```
            suma += notas[i];
```

```
        }
```

```
        return suma / N;
```

```
    }
```

Código en java:

```
import java.io.BufferedReader;

import java.io.InputStreamReader;

import java.io.IOException;

public class examenFINAL {

    public static void main(String[] args) throws IOException {

        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));

        int N;

        String nombre;

        double calif1, calif2;

        boolean valido;

        // 1. Lectura del número de alumnos

        do {

            System.out.print("Ingrese el numero de alumnos: ");

            try {

                N = Integer.parseInt(reader.readLine());

            } catch (NumberFormatException e) {

                N = 0;

            }

        } while (N <= 0);

        // Definición de arreglos (Vectores)

        String[] nombres = new String[N];

        double[] nota1 = new double[N];

        double[] nota2 = new double[N];

        double[] promediosAlumnos = new double[N];

        // 2. Proceso de captura de datos

        for (int i = 0; i < N; i++) {

            System.out.println("\n--- Alumno " + (i + 1) + " ---");

            // Validar Nombre

            do {

                System.out.print("Nombre: ");

                nombre = reader.readLine();

                valido = (nombre.length() > 0);

            } while (!valido);

        }

    }

}
```

```

        if (!valido) {

            System.out.println("Error: El nombre no puede estar vacio");

        }
    } while (!valido);

    nombres[i] = nombre;

// Validar Calificación 1

do {

    System.out.print("Calificacion 1 (1-100): ");

    try {

        calif1 = Double.parseDouble(reader.readLine());

        valido = (calif1 >= 1 && calif1 <= 100);

    } catch (NumberFormatException e) {

        valido = false;

    }

    if (!valido) {

        System.out.println("Error: Debe ser un numero entre 1 y 100");

    }

} while (!valido);

nota1[i] = calif1;

// Validar Calificación 2

do {

    System.out.print("Calificacion 2 (1-100): ");

    try {

        calif2 = Double.parseDouble(reader.readLine());

        valido = (calif2 >= 1 && calif2 <= 100);

    } catch (NumberFormatException e) {

        valido = false;

    }

    if (!valido) {

        System.out.println("Error: Debe ser un numero entre 1 y 100");

    }

} while (!valido);

nota2[i] = calif2;

```

```

        // Calcular promedio individual

        promediosAlumnos[i] = (nota1[i] + nota2[i]) / 2.0;
    }

    // 3. Mostrar Resultados

    System.out.println("\n-----");

    System.out.println("        RESULTADOS ");

    System.out.println("-----");

    System.out.println("Alumno \tNota1 \tNota2 \tPromedio");

    System.out.println("-----");

    for (int i = 0; i < N; i++) {

        System.out.println(nombres[i] + " \t" + nota1[i] + " \t" + nota2[i] + " \t" + promediosAlumnos[i]);

    }

    System.out.println("\n-----");

    System.out.printf("Promedio general del grupo: %.2f\n", promedioGeneral(promediosAlumnos, N));

    System.out.printf("Promedio de asignatura 1: %.2f\n", promedioAsignatura(nota1, N));

    System.out.printf("Promedio de asignatura 2: %.2f\n", promedioAsignatura(nota2, N));

    System.out.println("-----");

}

// --- Métodos (Funciones) ---

public static double promedioGeneral(double[] promedios, int N) {

    double suma = 0;

    for (int i = 0; i < N; i++) {

        suma += promedios[i];

    }

    return suma / N;

}

public static double promedioAsignatura(double[] notas, int N) {

    double suma = 0;

    for (int i = 0; i < N; i++) {

        suma += notas[i];

    }

    return suma / N;

}

}

```


pse int:

Algoritmo examenFINAL

Definir N, i Como Entero

Definir nombre Como Cadena

Definir calif1, calif2, suma Como Real

Definir valido Como Logico

Repetir

 Escribir "Ingrese el numero de alumnos: "

 Leer N

Hasta Que N > 0

Dimension nombres[N]

Dimension nota1[N]

Dimension nota2[N]

Dimension promediosAlumnos[N]

Definir nombres Como Cadena

Definir nota1, nota2, promediosAlumnos Como Real

Para i <- 1 Hasta N Con Paso 1 Hacer

 Escribir ""

 Escribir "--- Alumno ", i, " ---"

 Repetir

 Escribir "Nombre: "

 Leer nombre

 valido <- Longitud(nombre) > 0

 Si No valido Entonces

 Escribir "Error: El nombre no puede estar vacio"

 FinSi

 Hasta Que valido

 nombres[i] <- nombre

 Repetir

 Escribir "Calificacion 1 (1-100): "

 Leer calif1

 valido <- (calif1 >= 1 Y calif1 <= 100)

 Si No valido Entonces

Escribir "Error: Debe ser un numero entre 1 y 100"

FinSi

Hasta Que valido

nota1[i] <- calif1

Repetir

Escribir "Calificacion 2 (1-100): "

Leer calif2

valido <- (calif2 >= 1 Y calif2 <= 100)

Si No valido Entonces

Escribir "Error: Debe ser un numero entre 1 y 100"

FinSi

Hasta Que valido

nota2[i] <- calif2

promediosAlumnos[i] <- (nota1[i] + nota2[i]) / 2.0

FinPara

Escribir ""

Escribir "-----"

Escribir " RESULTADOS "

Escribir "-----"

Escribir "Alumno Nota1 Nota2 Promedio"

Escribir "-----"

Para i <- 1 Hasta N Con Paso 1 Hacer

Escribir nombres[i], " ", nota1[i], " ", nota2[i], " ", promediosAlumnos[i]

FinPara

Escribir ""

Escribir "-----"

Escribir "Promedio general del grupo: ", promedioGeneral(promediosAlumnos, N)

Escribir "Promedio de asignatura 1: ", promedioAsignatura(nota1, N)

Escribir "Promedio de asignatura 2: ", promedioAsignatura(nota2, N)

Escribir "-----"

FinAlgoritmo

Funcion promG <- promedioGeneral(promedios, N)

Definir promG, suma Como Real

Definir i Como Entero

suma <- 0

Para i <- 1 Hasta N Con Paso 1 Hacer

```
        suma <- suma + promedios[i]
```

```
    FinPara
```

```
    promG <- suma / N
```

```
FinFuncion
```

```
Funcion promA <- promedioAsignatura(notas, N)
```

```
    Definir promA, suma Como Real
```

```
    Definir i Como Entero
```

```
    suma <- 0
```

```
    Para i <- 1 Hasta N Con Paso 1 Hacer
```

```
        suma <- suma + notas[i]
```

```
    FinPara
```

```
    promA <- suma / N
```

```
FinFuncion
```